

栈

先进后出 先插入的元素后出来



队列

先进先出 先插入的元素先出来 $\leftarrow \underline{000} \leftarrow$

① 栈

int stk[N] tt = 0 从 st[0] 开始使用

stk 存数 tt 指向栈顶

入栈: $stk[++tt] = x$

出栈: $tt--$

判断栈空 if (tt > 0) 不空 else 空

取栈顶 $stk[tt]$

② 队列

int q[N] hh = 0 tt = -1

q 存数 hh 表示队头 tt 表示队尾

插入 $q[++tt] = x$

弹出 $hh++$

判空: if $hh \leq tt$ 不空 else 空

取队头 $q[hh]$

取队尾 $q[tt]$

828. 模拟栈

📖 题目

📝 提交记录

💬 讨论

📖 题解

📺 视频讲解

实现一个栈，栈初始为空，支持四种操作：

- `push x` - 向栈顶插入一个数 x ；
- `pop` - 从栈顶弹出一个数；
- `empty` - 判断栈是否为空；
- `query` - 查询栈顶元素。

现在要对栈进行 M 个操作，其中的每个操作 3 和操作 4 都要输出相应的结果。

输入格式

第一行包含整数 M ，表示操作次数。

接下来 M 行，每行包含一个操作命令，操作命令为 `push x`，`pop`，`empty`，`query` 中的一种。

输出格式

对于每个 `empty` 和 `query` 操作都要输出一个查询结果，每个结果占一行。

其中，`empty` 操作的查询结果为 `YES` 或 `NO`，`query` 操作的查询结果为一个整数，表示栈顶元素的值。

数据范围

$1 \leq M \leq 100000$,
 $1 \leq x \leq 10^9$

所有操作保证合法，即不会在栈为空的情况下进行 `pop` 或 `query` 操作。

输入样例：

```
10
push 5
query
push 6
pop
query
pop
empty
push 4
query
empty
```

输出样例：

```
5
5
YES
4
NO
```

难度：

简单

时/空限制：

1s / 64MB

总通过数：

81664

总尝试数：

117474

来源：

模板题

算法标签

▼

```
1 #include <stdio.h>
2 #include <string.h>
3
4 typedef long long LL;
5 const int N = 1e5 + 10;
6 int n;
7 int stk[N], tt = -1;
8
9 void push(int x) { // 插入
10     stk[++ tt] = x;
11     return ;
12 }
13
14 void pop() { // 弹出
15     tt --;
16     return ;
17 }
18
19 int empty() { // 判空
20     return tt == -1;
21 }
22
23 int query() { // 查询栈顶
24     return stk[tt];
25 }
26
27 int main() {
28     scanf("%d", &n);
29     char in[10];
30     int x;
31     for(int i = 0; i < n; i++) {
32         scanf("%s", in);
33         if(strcmp(in, "push") == 0) {
34             scanf("%d", &x);
35             push(x);
36         } else if(strcmp(in, "pop") == 0) {
37             pop();
38         } else if(strcmp(in, "empty") == 0) {
39             if(empty()) printf("YES\n");
40             else printf("NO\n");
41         } else {
42             printf("%d\n", query());
43         }
44     }
45
46     return 0;
47 }
48 }
```

r0tten

3302. 表达式求值

国 题目

提交记录

讨论

题解

视频讲解

给定一个表达式，其中运算符仅包含 `+, -, *, /`（加 减 乘 整除），可能包含括号，请你求出表达式的最终值。

注意：

- 数据保证给定的表达式合法。
- 题目保证符号 `-` 只作为减号出现，不会作为负号出现，例如，`-1+2`，`(2+2)*(-(1+1)+2)` 之类表达式均不会出现。
- 题目保证表达式中所有数字均为正整数。
- 题目保证表达式在中间计算过程以及结果中，均不超过 $2^{31}-1$ 。
- 题目中的整除是指向 0 取整，也就是说对于大于 0 的结果向下取整，例如 $5/3=1$ ，对于小于 0 的结果向上取整，例如 $5/(1-4)=-1$ 。
- C++和Java中的整除默认是向零取整；Python中的整除 `//` 默认向下取整，因此Python的 `eval()` 函数中的整除也是向下取整，在本题中不能直接使用。

输入格式共一行，为给定表达式。

输出格式共一行，为表达式的结果。

数据范围表达式的长度不超过 10^5 。

输入样例

(2+2)*(1+1)

输出样例

8

难度：中等

时/空限制：1s / 64MB

总通过数：60112

总尝试数：110576

来源：模板题 AcWing

算法标签

```
1 #include <stdio.h>
2
3 const int N = 1e5 + 10;
4 char exp[N];
5
6 int stk1[N], tt1 = -1; // 存储数字
7 char stk2[N]; int tt2 = -1; // 存储运算符
8
9 // 运算符优先级：
10 // ( : 0, + - : 1, * / : 2
11 // 每个运算符入栈二，要把优先级大于等于自己的运算符先弹出去计算
12
13 void cal(char c) { // 运算
14     // 注意，我们先取出来的是num2，再是num1，计算num1 c num2
15     int a = stk1[tt1--], b = stk1[tt1--];
16     if(c == '+') stk1[++ tt1] = b + a;
17     else if(c == '-') stk1[++ tt1] = b - a;
18     else if(c == '*') stk1[++ tt1] = b * a;
19     else stk1[++ tt1] = b / a;
20 }
21
22 int main() {
23     scanf("%s", exp);
24
25     int tmp = 0; // 存储数字
26     char c;
27     for(int i = 0; exp[i]; i++) {
28         if(exp[i] >= '0' && exp[i] <= '9') tmp = tmp * 10 + exp[i] - '0'; // 数字情况
29         else { // 运算符情况
30             if(tmp) { // 之前的部分是有效数字，加入栈1中
31                 stk1[++ tt1] = tmp;
32                 tmp = 0; // 清空数字
33             }
34
35             if(exp[i] == '(') stk2[++ tt2] = exp[i]; // 左括号直接入栈
36             else if(exp[i] == ')') {
37                 while(stk2[tt2] != '(') {
38                     c = stk2[tt2--];
39                     cal(c);
40                 } // 计算栈2中第一个左括号之前的所有运算符
41                 tt2--; // 弹出左括号
42             } // 运算符括号，运算符弹至出现右括号
43             else if(exp[i] == '*' || exp[i] == '/') {
44                 while(tt2 >= 0 && (stk2[tt2] == '*' || stk2[tt2] == '/')) {
45                     c = stk2[tt2--];
46                     cal(c);
47                 }
48                 stk2[++ tt2] = exp[i];
49             } // * / 把优先级大于等于自己的弹出去 (* /) 计算
50             else {
51                 while(tt2 >= 0 && stk2[tt2] != '(') {
52                     c = stk2[tt2--];
53                     cal(c);
54                 }
55                 stk2[++ tt2] = exp[i];
56             } // + - 把优先级大于等于自己的弹出去 (+ * / + -) 计算
57         }
58     }
59     if(tmp) stk1[++ tt1] = tmp; // 因为我们是用运算符号截断数字，所以最后还要再判断一下
60     while(tt2 >= 0) cal(stk2[tt2--]); // 弹出栈中剩余的运算符
61     printf("%d", stk1[tt1]); // 最后栈1中只会会有一个结果，就是运算答案
62
63     return 0;
64 }
```

中缀表达式求值：

两个栈 栈1：存数字

 栈2：存运算符和（

①遇到数字，直接入栈1

②遇到运算符： 1° （ 栈2

 2° ） 弹栈2的运算符并运算，

 直至弹出第一个（

3° +-*/ 把栈2中优先级 > 自身的弹出运算，自己再入栈

829. 模拟队列

📖 题目

📄 提交记录

💬 讨论

📖 题解

📺 视频讲解

实现一个队列，队列初始为空，支持四种操作：

- `push x` – 向队尾插入一个数 x ；
- `pop` – 从队头弹出一个数；
- `empty` – 判断队列是否为空；
- `query` – 查询队头元素。

现在要对队列进行 M 个操作，其中的每个操作 3 和操作 4 都要输出相应的结果。

输入格式

第一行包含整数 M ，表示操作次数。

接下来 M 行，每行包含一个操作命令，操作命令为 `push x`，`pop`，`empty`，`query` 中的一种。

输出格式

对于每个 `empty` 和 `query` 操作都要输出一个查询结果，每个结果占一行。

其中，`empty` 操作的查询结果为 `YES` 或 `NO`，`query` 操作的查询结果为一个整数，表示队头元素的值。

数据范围

$1 \leq M \leq 100000$ ，
 $1 \leq x \leq 10^9$ ，
所有操作保证合法，即不会在队列为空的情况下进行 `pop` 或 `query` 操作。

输入样例：

```
10
push 6
empty
query
pop
empty
push 3
push 4
pop
query
push 6
```

输出样例：

```
NO
6
YES
4
```

难度：

简单

时空限制：1s / 64MB

总通过数：71853

总尝试数：99297

来源：

模板题

算法标签

```
1 #include <stdio.h>
2 #include <string.h>
3
4 const int N = 1e5 + 10;
5 int n;
6 int q[N], hh = 0, tt = -1;
7
8 void push(int x) { // 入队操作
9     q[++ tt] = x;
10    return ;
11 }
12
13 void pop() { // 出队操作
14     hh ++;
15     return ;
16 }
17
18 int empty() { // 判空
19     return hh > tt;
20 }
21
22 int query() { // 查询队头元素
23     return q[hh];
24 }
25
26 int main() {
27     scanf("%d", &n);
28     char in[10]; int x;
29     for(int i = 0; i < n; i++) {
30         scanf("%s", in);
31         if(strcmp(in, "push") == 0) {
32             scanf("%d", &x);
33             push(x);
34         } else if(strcmp(in, "pop") == 0) {
35             pop();
36         } else if(strcmp(in, "empty") == 0) {
37             if(empty()) printf("YES\n");
38             else printf("NO\n");
39         } else {
40             printf("%d\n", query());
41         }
42     }
43
44     return 0;
45 }
```


单调栈

最常见应用：下一个更大的数

找到一组数中，每个数的左边（或右边）比它大且离它最近的数

找到每个数左侧比它小且最近的数

3 4 2 7 5 ✓

返回 -1 3 -1 2 2

考虑方式

① 暴力做法 双重 for 循环暴力 $O(n^2)$

② 单调栈优化

└──────────┘

$i \rightarrow$

i 向右扫描时，可用栈存左侧的所有元素

当我们到 i 位置时，判断 栈顶与 $A[i]$ 的大小

若栈顶元素更小，那上一个更小元素就是栈顶，否则 pop，直至栈顶更小或栈空

栈空说明左侧无更小元素，输出 -1

上述操作结束后， i 位置元素也入栈， $i \rightarrow$ 下一个元素



栈内元素从栈底到栈顶单调增

因为只要有逆序关系，就会被 pop 掉

val	7	2	2比7小，且对之后元素来说，
index	4	5	2更近，因此7永不可能用到

830. 单调栈

🏠 题目

📄 提交记录

💬 讨论

📖 题解

🎥 视频讲解

给定一个长度为 N 的整数数列，输出每个数左边第一个比它小的数，如果不存在则输出 -1 。

输入格式

第一行包含整数 N ，表示数列长度。

第二行包含 N 个整数，表示整数数列。

输出格式

共一行，包含 N 个整数，其中第 i 个数表示第 i 个数的左边第一个比它小的数，如果不存在则输出 -1 。

数据范围

$1 \leq N \leq 10^5$
 $1 \leq \text{数列中元素} \leq 10^9$

输入样例：

```
5
3 4 2 7 5
```

输出样例：

```
-1 3 -1 2 2
```

难度：简单

时/空限制：1s / 64MB

总通过数：132261

总尝试数：181327

来源：模板题

算法标签

```
1 #include <stdio.h>
2 typedef long long LL;
3 const int N = 1e5 + 10;
4 int n;
5 int stk[N], tt = -1;
6 int arr[N];
7
8 void push(int x) { // 插入
9     stk[++ tt] = x;
10    return ;
11 }
12
13 void pop() { // 弹出
14     tt --;
15     return ;
16 }
17
18 int empty() { // 判空
19     return tt == -1;
20 }
21
22 int query() { // 查询栈顶
23     return stk[tt];
24 }
25
26 int main() {
27     scanf("%d", &n);
28     for(int i = 0; i < n; i++) scanf("%d", &arr[i]);
29     for(int i = 0; i < n; i++) {
30         // while(!empty() && arr[query()] >= arr[i]) pop();
31         // 如果是输出下标，那我们栈中就存下标，元素直接到arr中取
32         // 如果栈中存数，我们还得再额外存这个数对应的下标，浪费空间
33         while(!empty() && query() >= arr[i]) pop();
34         // 始终保证栈中元素从栈底到栈顶是单调递增的
35         if(empty()) printf("-1 ");
36         else printf("%d ", query());
37         push(arr[i]);
38     }
39
40     return 0;
41 }
```

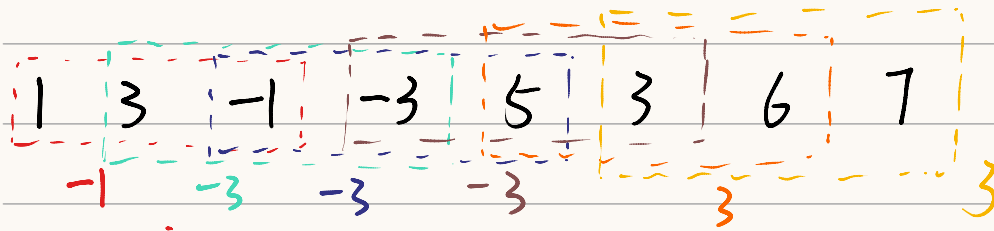
我们在实际写的
时候，可以不用把对栈
的操作写成函数形式
省时间。

每个元素只会进栈一次，最多出栈一次，因此时间复杂度为 $O(n)$

单调队列

常见应用：滑动窗口中的最值

一维数组，从左向右遍历，相邻的 k 个数组为一个窗口，求当前窗口最值



窗口可用队列维护

思路：暴力法：每一个窗口，都遍历窗口的所有元素 $O(kn)$

优化：思考单调性：队列中是否存在无用元素

$\begin{matrix} 3 & -1 & -3 & 5 \\ 0 & 1 & 2 & \end{matrix}$

$i=0$ ，队空， 3 入队 $i=1$ ， $-1 < 3$ ， 3 先从队尾弹出， -1 入队

$i=2$ $-1 > -3$ -1 先从队尾弹出， -3 入队 $i=4$ $-3 < 5$ ， 5 入

上述过程中队头始终为 $\min$ 。

弹出一定是从队尾弹而不是队头

队列中存的是下标而不是具体的值

这样就可以判断队头是否在当前窗口中

154. 滑动窗口

- 题目
- 提交记录
- 讨论
- 题解
- 视频讲解

给定一个大小为 $n \leq 10^6$ 的数组。

有一个大小为 k 的滑动窗口，它从数组的最左边移动到最右边。

你只能在窗口中看到 k 个数字。

每次滑动窗口向右移动一个位置。

以下是一个例子：

该数组为 `[1 3 -1 -3 5 3 6 7]`， k 为 `3`。

窗口位置	最小值	最大值
[1 3 -1] -3 5 3 6 7	-1	3
1 [3 -1 -3] 5 3 6 7	-3	3
1 3 [-1 -3 5] 3 6 7	-3	5
1 3 -1 [-3 5 3] 6 7	-3	5
1 3 -1 -3 [5 3 6] 7	3	6
1 3 -1 -3 5 [3 6 7]	3	7

你的任务是确定滑动窗口位于每个位置时，窗口中的最大值和最小值。

输入格式

输入包含两行。

第一行包含两个整数 n 和 k ，分别代表数组长度和滑动窗口的长度。

第二行有 n 个整数，代表数组的具体数值。

同行数据之间用空格隔开。

输出格式

输出包含两个。

第一行输出，从左至右，每个位置滑动窗口中的最小值。

第二行输出，从左至右，每个位置滑动窗口中的最大值。

数据范围

$$1 \leq n \leq 10^6,$$
$$1 \leq k \leq n$$

输入样例：

```
8 3
1 3 -1 -3 5 3 6 7
```

输出样例：

```
-1 -3 -3 -3 3 3
3 3 5 5 6 7
```

难度：

简单

时/空限制：1s / 64MB

总通过数：143255

总尝试数：277616

来源：

《算法竞赛进阶指南》

模板题

《信息学奥赛一本通》算法提高篇

算法标签▼

```

1 #include <stdio.h>
2
3 const int N = 1e6 + 10;
4 int n, k;
5 int arr[N]; // 存储读入的数据
6 int q1[N], hh1 = 0, tt1 = -1; // 队列内单调递增（这里存的是下标）
7 int q2[N], hh2 = 0, tt2 = -1; // 单调递减
8 int MIN[N], MAX[N], idx = 0;
9
10 int main() {
11     scanf("%d%d", &n, &k);
12     for(int i = 0; i < n; i++) scanf("%d", &arr[i]);
13     int i = 0, j = 0;
14
15     while(j < n) {
16         // 首先扩大窗口，处理队列内单调性
17         // 这里是从队尾弹出，
18         // 满足q1内单调增，这样队头是MIN
19         // q2单调减，队头是MAX
20         // 这里从队尾弹出而不是队头，只有这样才能保证单调性！
21         while(hh1 <= tt1 && arr[q1[tt1]] >= arr[j]) tt1--;
22         while(hh2 <= tt2 && arr[q2[tt2]] <= arr[j]) tt2--;
23         q1[++tt1] = j; q2[++tt2] = j;
24         j++;
25
26         // 因为j ++, 所以这里是j - i而不是j - i + 1
27         // 这里的j是下次要放入的元素
28         // j 0 -> n - 1
29         if(j - i == k) {
30             // 这里表明窗口已经长k，要缩小窗口，下一次窗口才能再次到k
31             MIN[idx] = arr[q1[hh1]];
32             MAX[idx++] = arr[q2[hh2]];
33             if(q1[hh1] == i) hh1++; // 如果缩小的正好是队头，就要弹出
34             if(q2[hh2] == i) hh2++;
35             i++;
36         }
37     }
38
39     for(int i = 0; i < idx; i++) printf("%d ", MIN[i]); printf("\n");
40     for(int i = 0; i < idx; i++) printf("%d ", MAX[i]);
41
42     return 0;
43 }

```

这里为了保证队列的单调性，
必须是从尾部弹出

例： 1 10 5

若从头弹 1入 10入 5入

1 10 5 不单调

从尾弹 1入 10入 10出 5入

1 5 单调

头部的弹出只是为了队列中的
内容是在当前窗口中的

```

47 #include <stdio.h>
48
49 const int N = 1e6 + 10;
50 int n, k;
51 int arr[N]; // 存储读入的数据
52 int q1[N], hh1 = 0, tt1 = -1; // 队列内单调递增（这里存的是下标）
53 int q2[N], hh2 = 0, tt2 = -1; // 单调递减
54 int MIN[N], MAX[N], idx = 0;
55
56 int main() {
57     scanf("%d%d", &n, &k);
58     for(int i = 0; i < n; i++) scanf("%d", &arr[i]);
59     int i = 0, j = -1;
60
61     while(j < n - 1) {
62         // 首先扩大窗口，处理队列内单调性
63         // 这里是从队尾弹出，
64         // 满足q1内单调增，这样队头是MIN
65         // q2单调减，队头是MAX
66         // 这里从队尾弹出而不是队头，只有这样才能保证单调性！
67         j++;
68         while(hh1 <= tt1 && arr[q1[tt1]] >= arr[j]) tt1--;
69         while(hh2 <= tt2 && arr[q2[tt2]] <= arr[j]) tt2--;
70         q1[++tt1] = j; q2[++tt2] = j;
71
72         // 这里的j是这次放入的元素
73         // j从-1 -> n - 2
74         if(j - i + 1 == k) {
75             // 这里表明窗口已经长k，要缩小窗口，下一次窗口才能再次到k
76             MIN[idx] = arr[q1[hh1]];
77             MAX[idx++] = arr[q2[hh2]];
78             if(q1[hh1] == i) hh1++; // 如果缩小的正好是队头，就要弹出
79             if(q2[hh2] == i) hh2++;
80             i++;
81         }
82     }
83
84     for(int i = 0; i < idx; i++) printf("%d ", MIN[i]); printf("\n");
85     for(int i = 0; i < idx; i++) printf("%d ", MAX[i]);
86
87     return 0;
88 }

```

一两种不同

看for单轮结束后j的位置

①: j在本轮存入的下一位

$\therefore j: 0 \rightarrow n-1$

② j就是本轮存入的

$\therefore j: -1 \rightarrow n-2$

因为j先++再执行存入

若从0开始第一个会漏

